

Informe - G-115

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

14 de Junio de 2026

1. Introducción	1
2. Modelo Computacional	1
2.1. Dominio	1
2.2. Lenguaje	2
3. Implementación	4
3.1. Frontend	4
3.2. Backend	4
3.3. Código base	5
3.4. Dificultades encontradas	5
4. Futuras extensiones	5
5. Conclusiones	5
6. Bibliografía	6

1. Introducción

El objetivo del presente trabajo fue aplicar los conocimientos adquiridos a lo largo de la materia Autómatas, Teoría de Lenguajes y Compiladores mediante el desarrollo de un compilador para un Domain Specific Language asociado a variables financieras personales.

El compilador se desarrolló en dos etapas: una centrada en el frontend del mismo (análisis léxico y sintáctico) y una centrada en el backend (análisis semántico y generación de código). Previamente, se dio una fase de diseño en la que se determinó el dominio del lenguaje.

Como base para el desarrollo del proyecto, se usó como base un proyecto provisto por la cátedra.

2. Modelo Computacional

2.1. Dominio

El dominio del lenguaje son variables asociadas a las finanzas personales, como por ejemplo ingresos, egresos, deudas, bienes inmuebles y objetivos financieros. Cada una de estas variables comparte aspectos en común con las otras (por ejemplo, todas tienen un valor monetario y una moneda), pero también tienen propiedades específicas de dichas variables (como por ejemplo una fecha de vencimiento para una deuda o una periodicidad para un ingreso).

2.2. Lenguaje

El lenguaje posee las siguientes características:

Declaración de variables:

Se pueden declarar distintas variables financieras, de la siguiente manera:

Tipo de variable	Estructura de la declaración	Ejemplo
Ingreso	income <i>nombre</i> as { value = <i>valor</i> , currency = <i>código de moneda</i> , periodicity = <i>periodicidad</i> };	income sueldo as { value = 150000.00, currency = ARS, periodicity = monthly };
Gasto	expenses <i>nombre</i> as { value = <i>valor</i> , currency = <i>código de moneda</i> , periodicity = <i>periodicidad</i> , category = " <i>categoría</i> " };	expenses alquiler as { value = 80000.00, currency = ARS, periodicity = monthly, category = "Vivienda" };
Bien físico	asset <i>nombre</i> as { value = <i>valor</i> , currency = <i>código de moneda</i> };	asset laptop as { value = 2000.00, currency = USD };
Deuda	debt <i>nombre</i> as { value = <i>valor</i> , currency = <i>código de moneda</i> , category = " <i>categoría</i> ", deadline = " <i>fecha dd-mm-yyyy</i> ", interest = <i>porcentaje de interés</i> , minPayment = <i>valor de pago mínimo</i> };	debt tarjetaCredito as { value = 200000.00, currency = ARS, category = "Deudas", deadline = "01-12-2026", interest = 5%, minPayment = 20000.00 };
Objetivo	goal <i>nombre</i> as { value = <i>valor</i> , currency = <i>código de moneda</i> , deadline = " <i>fecha dd-mm-yyyy</i> " };	goal vacaciones as { value = 300000.00, currency = ARS, deadline = "01-07-2027" };

Consultas:

Se pueden consultar variables predefinidas y en algunos casos operar con ellas.

Variable	Estructura de la consulta	Ejemplo
Ingresos totales	totalIncome as { from "fecha desde dd-mm-yyyy", up "fecha hasta dd-mm-yyyy" };	totalIncome as { from "01-01-2026", up "31-12-2026" };
Gastos totales	totalExpenses as { from "fecha desde dd-mm-yyyy", up "fecha hasta dd-mm-yyyy" };	totalExpenses as { from "01-01-2026", up "31-12-2026" };
Gastos totales por categoría	totalExpenses as { from "fecha desde dd-mm-yyyy", up "fecha hasta dd-mm-yyyy", category = "categoría" };	totalExpenses as { from "01-01-2026", up "31-12-2026", category = "Vivienda" };
Categoría en la que más se gasta	maxCategory;	maxCategory;
Categoría en la que menos se gasta	minCategory;	minCategory;
Balance	balance nombre as { from "fecha desde dd-mm-yyyy", up "fecha hasta dd-mm-yyyy" };	balance resumenAnual as { from "01-01-2026", up "31-12-2026" };
Plan de pago para una deuda en cierta cantidad de cuotas	plan for nombreDeuda in numeroCuotas;	plan for tarjetaCredito in 6;
Cambio de moneda para una variable	exchange nombreVariable códigoMoneda;	exchange sueldo USD;
Cambio de periodicidad para una variable	change-periodicity nombreVariable periodo;	change-periodicity sueldo yearly;
Exportar datos a CSV	export dato;	export resumenAnual;
Generación de gráficos circulares	circleGraphic dato;	circleGraphic totalExpenses;

Para operar con estas variables, se debe usar `derivatedData`. Por ejemplo:

```
derivatedData disponible = totalIncome as {  
  from "01-01-2026",  
  up "31-12-2026"  
} - totalExpenses as {  
  from "01-01-2026",  
  up "31-12-2026"  
};
```

3. Implementación

3.1. Frontend

En esta etapa se trabajó en dos partes. En primer lugar, se realizó una etapa de análisis léxico, en la que se trabajó con el analizador léxico Flex para generar un stream de tokens que posteriormente fue transformado en un árbol de sintaxis abstracta (AST) usando Bison.

Para el análisis léxico, se definieron los símbolos terminales (tokens) correspondientes a la gramática del lenguaje. Algunos ejemplos son INCOME, EXPENSES, etcétera. Posteriormente, en la etapa de análisis sintáctico se definieron las producciones libres de contexto correspondientes a la gramática.

3.2. Backend

En primer lugar, se realizó una etapa de análisis semántico para validar que el contenido del árbol de sintaxis abstracta se correspondiera con el dominio del lenguaje. En esta etapa se incorporó una tabla de símbolos para verificar que no se pudieran declarar variables de manera duplicada, no se pudieran realizar acciones como exportar un dato que no hubiera sido declarado o realizar operaciones entre tipos incompatibles (como por ejemplo sumar datos que no fueran numéricos). Asimismo se agregaron validaciones para evitar divisiones por cero o declarar una fecha de finalización posterior a una fecha de inicio. Se resolvió no usar un stack para scopes y rechazar cualquier variable repetida sin mayor análisis, dada la definición del lenguaje.

En segundo lugar, se realizó la etapa de generación de código, en la que a partir del contenido del árbol de sintaxis abstracta se armó un código Java. Para esto, se tiene un código base que incluye un archivo `pom.xml` y varias clases Java que simplemente se copian en la salida (por defecto se genera una carpeta `output`) y simplemente se genera el contenido de la clase `Main`.

3.3. Código base

Como se mencionó anteriormente, la salida del compilador consiste en un proyecto Java, el cual incluye un archivo pom.xml para gestionar la instalación de librerías necesarias para la ejecución del mismo mediante Maven.

Se decidió esta salida al considerar que era la mejor manera de manejar a las variables económicas, considerando que como se mencionó al principio las mismas comparten ciertas propiedades y se diferencian en otras. Para esto se tienen clases abstractas como `EconomicVariable` y `SoftEconomicVariable` que sirven para modelar las propiedades como valor y moneda y clases hijas de estas como `Asset`, `Debt`, `Expense`, `Goal`, `Income` que agregan sus comportamientos propios. La clase `UtilFunctions` incluye funciones que son utilizadas por la clase `Main` generada para resolver las distintas consultas presentes en el código de entrada (como obtener el total de ingresos, por ejemplo).

Para la gestión de gráficos se utilizó la librería `XChart` y para el manejo de valores para la conversión de monedas se utilizó el API de `Coinbase`, previa consulta con la cátedra. Gracias a esta última, se pueden usar valores en tiempo real de los tipos de cambio.

3.4. Dificultades encontradas

Se nos presentaron dificultades principalmente para encontrar donde es necesario validar que no se esté intentando incorporar código que no es válido (como el caso de fechas de finalización posteriores a las fechas de inicio). Es probable que hayan quedado validaciones pendientes que puedan ser parte de futuras extensiones del proyecto.

Asimismo, se nos presentaron dificultades a la hora de elegir la salida del proyecto. Originalmente se planeaba que la misma fuera un código Python u Octave, considerando que ambos lenguajes son usados con mayor frecuencia para tareas estadísticas y ambos permiten manejar gráficos con mayor facilidad. Sin embargo, notamos que sería más fácil trabajar en un lenguaje conocido por nosotros y además resultaba beneficioso que fuera orientado a objetos.

4. Futuras extensiones

Como se mencionó, es probable que el compilador aún acepte código inválido y falten validaciones. En este punto se encuentra una potencial mejora para el proyecto.

Asimismo, en la etapa de backend se terminó dejando de lado el uso de operadores relacionales, al no encontrarles una aplicación práctica en el proyecto. En el futuro, se puede buscar la forma de incorporarlos.

5. Conclusiones

Como conclusiones nos llevamos que el proyecto fue una forma práctica de aplicar los conceptos adquiridos a lo largo de la materia. Asimismo fue una gran oportunidad para entender la importancia de cada fase (análisis léxico, análisis sintáctico, análisis semántico y generación de código) en el desarrollo de compiladores.

6. Bibliografía

Knowm (2026). XChart. GitHub. <https://github.com/known/XChart>

Coinbase (2026). Data API - Exchange rates. Coinbase Developer Documentation. <https://docs.cdp.coinbase.com/coinbase-business/track-apis/exchange-rates>